



DRISHTI

ENTERPRISE TECHNICAL SPECIFICATION

Drishti — Technical Requirements Document (TRD)

Defensive Cybersecurity Platform — Complete Architecture & Reference Manual



Graph-Powered Attack Paths



Live Watch Agent



Dollar Risk Quantifier



AI Remediation Guardrails



URL Trust Scoring

PLATFORM VERSION

v0.2 Enterprise Edition

RELEASE DATE

August 2026

ARCHITECTURE TIER

Full-Stack Multi-Tenant



RESTRICTED // INTERNAL ARCHITECTURE & TECHNICAL SPECIFICATION // FOR AUTHORIZED USE ONLY

Drishti — Technical Requirements Document (TRD)

Reverse-engineered from source. Formulas, coefficients, and contracts below are transcribed from the actual implementation in `server/app/services/`.

1. Technology stack

Layer	Technology	Notes
Backend framework	FastAPI	ASGI, lifespan bootstrap, dependency-injected auth
ORM	SQLAlchemy 2 (typed <code>Mapped[...]</code>)	UUID PKs as 36-char strings (Postgres + SQLite portable)
DB	PostgreSQL (Docker) / SQLite (local, tests)	driver pinned to <code>postgresql+psycopg://</code> (psycopg v3)
Risk engine	NetworkX DiGraph	Dijkstra, betweenness centrality, Yen's k-shortest paths
Validation	Pydantic v2 + <code>pydantic-settings</code>	request/response schemas; env-driven config
Auth	python-jose (JWT HS256) + bcrypt	access/refresh tokens, <code>token_version</code> invalidation
AI	Groq (Llama 3.3 70B, default) / Anthropic (Claude)	backend-only; <code>AI MOCK</code> for keyless demo
Scanning	nmap subprocess (<code>-sV</code>) + NVD /Vulners CVE APIs	consent-gated, RFC1918-only
Frontend	React 18 + Vite + TypeScript	code-split routes
Graph UI	React Flow (<code>reactflow</code> 11)	attack map; <code>d3-force</code> for the live force map
Data fetching	TanStack Query v5	cache + invalidation on mutations
Charts	Recharts	dashboard/report
Client state	Zustand	toasts, graph store
Styling	Tailwind CSS + <code>framer-motion</code>	SOC-blue design system
Runtime	Python 3.11+, Node (Vite dev on <code>:5173</code> , API on <code>:8000</code>)	

2. Backend application structure

`server/app/main.py` assembles the app:

- **Lifespan bootstrap** (`_bootstrap`): `create_all` tables → `reconcile_columns` (additive migration) → backfill device subnets → seed org identity (or Acme sample if `DEMO_SEED=1`). Idempotent; only seeds when no org exists. All wrapped in try/except so a seed failure never blocks boot.

- **Autoscan scheduler** started on boot (skipped under pytest). No-op until a user enables it per-org.
- **Convenience agent** auto-started in `conn` mode on boot (skipped under pytest).
- **Middleware stack** (outermost → innermost): `MaxBodySizeMiddleware` (413 on oversized body, streaming, pre-auth) → `structured_log` (per-request JSON log with request id + latency) → `CORSMiddleware` (allow-list + `chrome-extension://` regex for the Web Guard extension).
- **14 routers** mounted under `/api/*` (+ health at root). See [API_REFERENCE.md](#).

Layering: **router → service → model**. Routers do auth + shape; services hold all business logic and are mostly pure/DB-bound; models are SQLAlchemy only. The risk engine (`risk_engine.py`, `attack_paths.py`, `impact.py`) is **pure functions over a NetworkX graph** — no HTTP, no ORM writes (`recompute.py` persists).

3. Risk Intelligence Engine

NOTE

Source: `services/risk_engine.py`, `services/attack_paths.py`, `services/impact.py`. All coefficients live in one `RiskConfig` dataclass so the model is explainable in a sentence.

3.1 Graph construction (`build_engine`)

1. Add a synthetic `INTERNET` entry node.
2. Add every real asset as a node.
3. Add **exposure edges** `INTERNET → asset` for each `internet_facing` asset.
4. Add the org's declared connections (`network` / `admin` / `trust` / `exposure`).
5. Normalize business value across real assets to `0..1`.
6. Compute edge weights.

3.2 Edge weight (`_compute_edge_weights`)

For edge `u → v` with destination `dest`:

TEXT

```
ease = clamp(0.6 * dest.max_exploitability + 0.4 * (dest.max_cvss / 10)) # ease_of_compromise
weight = RELATION_BASE[relation] + (1.0 - ease)
```

Lower weight = easier hop. `RELATION_BASE = {exposure:0.1, network:0.2, trust:0.25, admin:0.15}`. A present vulnerability raises `ease`, lowering weight; resolving it raises the weight back — this is what makes *"mark resolved → exposure drops"* work.

3.3 Per-hop traversal ease (hop_ease) — for likelihood

TEXT

```
hop_ease(u,v) = clamp( max( ease_of_compromise(v), RELATION_EASE[relation] ) )
RELATION_EASE = {exposure:0.5, network:0.4, trust:0.45, admin:0.5}
```

The relation floor means a foothold can traverse a link even without a fresh vuln; a vuln-driven node ease dominates when present.

3.4 Node risk score 0–100 (compute_node_scores)

TEXT

```
risk = 100 · ( 0.30·exploit + 0.25·reach + 0.20·centrality + 0.15·value + 0.10·crit )
```

- `exploit` = `ease_of_compromise(node)`
- `reach` = `1/(1+dist)` from Dijkstra shortest path from INTERNET; **floored to 0.5** if reachable at all
- `centrality` = betweenness centrality (weighted, normalized), scaled to the graph max
- `value` = min-max-normalized business value
- `crit` = `CRIT_FACTOR[criticality]` = `{low:0.25, medium:0.5, high:0.75, critical:1.0}`

Weights (`w_exploit ... w_crit`) sum to 1.0.

3.5 Attack-path enumeration (enumerate_paths)

- **Targets** (`find_targets`) = crown-jewel zone **OR** `critical` criticality **OR** top-decile business value.
- For each target with a path from INTERNET: **Yen's** `shortest_simple_paths` (increasing weight).
- **Bounds:** `max_hops=6`, `paths_per_target=5`, `MAX_CANDIDATES_PER_TARGET=500`, global `top_k=25`. Never enumerate the unbounded simple-path set.
- **Path risk 0–100** (`_score_path`):

TEXT

```
likelihood = [] hop_ease over the path (clamped 0.001..0.999)
path_risk = 100 · ( 0.45·likelihood + 0.30·target_value + 0.15·target_crit + 0.10·(1 - weight_no
```

- Ranked by `path_risk` desc; deterministic tie-break (`hop_count`, `target_id`).

3.6 Blast radius (blast_radius)

`nx.descendants(graph, node)` minus INTERNET = every asset reachable if that node is compromised. Value = sum of downstream business value.

3.7 Business impact (\$) — `impact.py`

TEXT

```
path_impact = likelihood * asset_value * IMPACT_MULTIPLIER[type] + likelihood * breach_cost_base
```

- `IMPACT_MULTIPLIER` = {database:1.0, cloud:0.8, webapp:0.7, server:0.6, firewall/router:0.5, iot:0.4, workstation:0.3}
- `breach_cost_base` default **\$500,000** (env `BREACH_COST_BASE`).
- **Total exposure** = Σ over top paths of the **max impact per unique target** (dedup so one target isn't double-counted).
- `id_key(path)` = stable `target:hops:node>node>...` key for impact lookup.

3.8 Recompute orchestration (`recompute.py`)

Per-org, idempotent, triggered on ingest / finding resolve / asset edit:

1. (Postgres) take a `pg_advisory_xact_lock` on the org id to serialize concurrent recomputes.
2. `load_engine(db, org)` → build graph from DB.
3. `compute_node_scores` → cache `risk_score` + `blast_radius_count` per asset.
4. Persist recomputed edge weights back to `connections`.
5. Delete + rewrite cached `attack_paths` + `attack_path_steps`.
6. Compute `impact_usd` per path; store.
7. Record `_LAST_STATS[org]` (`nodes/edges/paths/recompute_ms/top_path_risk`) for `/api/stats`.

4. Ingestion pipeline (`services/ingest.py`)

`POST /api/ingest` (agent-token auth, rate-limited 60/min burst 20):

- Validates `IngestPayload` (host + services + vulnerabilities + connectivity); `org_slug` must match the agent's org.
- **Upsert asset** idempotently by `(org, ip)`, fallback to hostname if IP shifted; race-safe via `begin_nested` + `IntegrityError` adoption.
- Criticality is **never downgraded** from an agent hint (`_CRIT_RANK` compare). DMZ zone-hint → `internet_facing=True` for new assets.
- **Replace services** (prune vanished ports); **upsert findings** (CVE-keyed, else title+severity).
- **Reconciliation**: any still-open finding **absent** from the new scan → auto-`resolved` (mirrors service pruning). Operator-`resolved` / `accepted` findings stay closed on re-ingest.
- Records a `Scan` history row → triggers `recompute_org` → commit.

5. AI orchestration (`services/ai/`)

- **Provider-agnostic** `client.generate(system, user_json, mock_key, fallback, schema)`; provider = Groq (default) or Anthropic. Backend is the **only** LLM caller — the frontend never holds a key.

- **Remediation** (`/api/ai/remediate`): assembles real finding context (asset/service/vuln/zone) → model or mock → validates → persists a `Remediation`. Cached per finding+kind; `regenerate=true` forces a new call.
- **Kinds**: `ansible` (default) / `shell` / `cloud_cli` / `manual`. When the model is unavailable, a **deterministic template** builds a real fix referencing the actual hostname + CVE, correctly shaped per kind (e.g. cloud-CLI keeps an internet-facing port public behind a WAF instead of breaking it).
- **Offensive-output guard** (`_guard_offensive`): scans model **output** for markers (`reverse shell`, `weaponize`, `exfiltrate`, `ransomware`, ...); a hit → refuse. Deliberately not `exploit` / `payload` / `malware` (those appear in legitimate CVE text). No input-side guard — describing a real vuln is legitimate defensive context.
- **Impact** (`/api/ai/impact`): AI narrates the dollar figure but the endpoint **overwrites** its number with the engine's `impact_usd` — the model can explain the number, never change it.
- **Predict** (`/api/ai/predict`): forward-looking risk narrative.
- All AI endpoints are user-JWT auth + rate-limited (20/min, burst 6).

6. URL Trust Analyzer (`services/urltrust/`)

Transparent two-part scoring (`scoring.py`):

1. **Weighted base** over **only evaluated** signals (`pass` / `warn` / `fail`), renormalized over their weights so an unavailable signal or unconfigured provider neither tanks nor inflates the result. Nothing evaluable → neutral **50.0**, not a fabricated verdict.
 2. **Hard caps** — a single serious red flag ceilings the final score (caps only ever *lower*): `safe_browsing fail` → **15**, `virustotal fail` → **20**, embedded creds → 30, no-DNS/punycode/@ → 40, raw-IP host → 50, invalid TLS → 50, plain HTTP → 74; warns: brand-lookalike → 55, VT-suspicious → 60.
- **Bands**: `Trusted ≥ 75`, `Caution ≥ 40`, else `High Risk`.
 - Optional providers (Google Safe Browsing, VirusTotal) report `configured:false` and contribute nothing when unkeyed.
 - Signals: HTTPS, TLS validity, DNS resolution, punycode/homograph, `@`-obfuscation, raw-IP host, embedded credentials, brand-lookalike, threat-feed hits.

7. Deep Scan (`services/deepscan/`)

- **Consent-gated**: `consent:true` required (else 422).
- **Scope-gated**: target must be **private/LAN (RFC1918)**, not loopback/link-local (blocks `169.254.169.254` metadata endpoint). Public IPs refused (422). Range scans require a private CIDR, `/22` max (≤1024 addrs).
- **Real nmap** `-sV` subprocess (top-1000 ports; timeouts: 120 s single, 300 s per range-batch, 60 s discovery sweep). Batch of ≤32 hosts, hard ceiling 256 total.
- **Real CVE lookup** (`cve_lookup`): NVD free REST API by default (~5 req/30 s unkeyed, ~50 keyed) or Vulners if `VULNERS_KEY`. A lookup that can't run → `available:false` + truthful reason (never a fabricated CVE).

- Results feed `integration.apply_scan` → creates/updates an `Asset` + `Service`s + findings → one engine recompute → cross-host paths form on real data.
- Persisted as `DeepScan` rows (full `result_json`) so the UI replays the last scan per asset without re-scanning.
- **Autoscan** (`services/autoscan.py`): per-org scheduler (`AutoScanConfig`, default 420 s). Always scans this host; scans the rest of the subnet only when `scan_subnet` is explicitly enabled (authorization affirmed). Round-robin cursor over devices.

8. Network-config audit (`services/netconfig/`)

- **Detectors** (`detectors.py`) for **DMZ / NAT / DHCP** over observed topology + declared config.
- **Honesty model**: every finding is `status ∈ {real, unknown, passed}` and `source ∈ {observed, declared}`, and cites concrete `evidence`. Missing evidence → `unknown` (never a fabricated finding, never a false `passed`). Only `real` findings carry a severity that maps into the engine as `Vulnerability` + `AssetVulnerability` rows.
- **Live-network behavior**: DHCP inferred from the live gateway; **flat-network / no-DMZ** raised from the live device sweep (medium, `real`); NAT boundary-OK passed check when a gateway is present with no exposed sensitive ports.
- Transparent severity → engine model: `{critical:(9.0,0.85), high:(7.5,0.65), medium:(5.0,0.45), low:(3.0,0.25)}`.
- Sensitive ports never exposed to the internet: SSH/Telnet/RDP/PostgreSQL/MySQL/MSSQL/Redis/Mongo/Elastic/VNC.
- Persisted as `NetconfigAnalysis` (full `result_json` + `real_findings` count).

9. Live threat detection (`services/live_threats.py`)

Pure detector `detect_threats(devices, domains, now)` over the live inventory:

Threat	Signature	MITRE
<code>arp_spoof</code>	one IP mapped to ≥ 2 MACs (spoofed set suppresses rogue/service on that IP)	T1557 · Adversary-in-the-Middle
<code>rogue_device</code>	device first-seen ≤ 10 min ago	T1200 · Hardware Additions
<code>risky_service</code>	exposed port / CVE from a deep scan	T1210 · Exploitation of Remote Services
<code>malicious_domain</code>	HIGH -Risk domain observation (C2)	T1071 · Application-Layer Protocol
Emits <code>NetworkThreat</code> (kind/severity/title/detail/device/evidence/recommendation/mitre). <code>network_threats(db, org)</code> is the DB adapter. <code>inject_demo</code> / <code>clear_demo</code> add/remove a clearly-labeled synthetic intruder (<code>DEMO-ATTACK</code>, MAC <code>de:ad:be:ef:*</code>, domain <code>secure-paypal-login.drishti-demo.test</code>) for the on-map demo. Threats set <code>threat*</code> / <code>mitre</code> fields on graph nodes so the Attack Map lights up.		

10. Auth & security internals

NOTE

Full treatment in [SECURITY_MODEL.md](#). Summary:

- **JWT HS256**, {sub, org_id, type, token_version, iat, exp}; access 15 min, refresh 7 days.
- **bcrypt** with a sha256 pre-hash (avoids the 72-byte bcrypt limit for long/UTF-8 passwords).
- **Timing-safe login**: a precomputed `DUMMY_PASSWORD_HASH` is verified when the email is unknown so login latency doesn't leak account existence.
- `token_version` embedded in every token; bumped on password change → invalidates all prior tokens.
- **Agent tokens** hashed (sha256) and looked up by hash equality; separate from user JWTs.
- **Rate limiting**: in-memory token buckets — ingest 60/min (burst 20) per agent, AI 20/min (burst 6) per user; stale-bucket eviction.

- **Body-size:** streaming `MaxBodySizeMiddleware` rejects > 1 MB pre-auth (413).
- **Fail-closed config:** refuses to boot on the default `change-me` JWT secret unless `APP_ENV ∈ {local, dev, test}`.

11. Data persistence & schema evolution

- **UUID PK** = 36-char string (`uuid_pk()`), portable across Postgres + SQLite; SQLite FK pragma enabled on connect.
- **No Alembic** (deliberate v1 gap). `create_all` makes missing *tables*; `reconcile_columns` (`db_init.py`) **adds** missing *columns* additively (nullable or defaulted only; add-nullable → backfill → tighten to NOT NULL on non-SQLite). Never drops/renames/retypes — destructive changes require recreating the DB.
- Full model in [DATA_MODEL.md](#).

12. Frontend architecture

- **Routing** (`App.tsx` / `ProtectedApp.tsx`): public `/` (landing), `/login`, `/signup`; protected `/app/*` behind `RequireAuth`. Heavy halves (landing, auth, authed app w/ React Flow + Recharts) are **lazy-loaded** code-split chunks.
- **Screens** (`src/features/*`): dashboard, graph (Attack Map), live (Live Watch), paths, findings, assets, report, url-analyzer, remediation console, settings, onboarding.
- **Data:** TanStack Query (retry 1, no refetch-on-focus, 5 s stale) with `qc.invalidateQueries()` after mutations (e.g. Recompute) so views refresh. Zustand for toasts + graph store.
- **Resilience:** nested React error boundaries — a React Flow crash on the Attack Map doesn't take down the routed view.
- **Live map fit:** `ForceMap` auto-fits via an effect keyed on node **count** (decoupled from poll churn) so nodes never drift off-screen.

13. Testing

- **Backend:** pytest — ~30 modules (`test_read_service`, `test_impact`, `test_paths`, `test_blast_radius`, `test_scoring`, `test_ingest`, `test_recompute`, `test_live_devices`, `test_live_threats`, `test_netconfig`, `test_deepscan`, `test_urltrust`, `test_auth_security`, `test_contracts`, ...). Autoscan/agent subprocess skipped under pytest.
- **Frontend:** vitest + Testing Library (`ForceMap.test`, `breachSim.test`, `graphStore.test`, `RiskPill.test`, `format.test`, `client.test`).
- **Smoke** (`tests/smoke.py`, `make smoke`): asserts the live exposure drop `$902,900 → $702,900` end-to-end.

14. Configuration matrix (env)

Var	Default	Purpose
APP_ENV	(empty = treated as deployment)	gates the insecure-JWT-secret allowance
DATABASE_URL	sqlite:///./drishti.db	normalized to postgresql+psycpg:// for managed PG
JWT_SECRET	change-me	must be set in non-dev or boot refuses
JWT_ACCESS_MINUTES / JWT_REFRESH_DAYS	15 / 7	token lifetimes
CORS_ORIGINS	http://localhost:5173	comma list; chrome-extension://* allowed by regex
AI_PROVIDER	groq	or anthropic
AI_MODEL	(tracks provider)	llama-3.3-70b-versatile / claude-sonnet-5
AI MOCK	false	true = keyless canned/deterministic fixes
GROQ_API_KEY / ANTHROPIC_API_KEY	—	provider key (backend only)
GOOGLE_SAFE_BROWSING_KEY / VIRUSTOTAL_KEY	—	optional URL-trust providers
NVD_API_KEY / VULNERS_KEY	—	optional CVE-lookup keys
BREACH_COST_BASE	500000	\$ impact base constant
INGEST_MAX_BYTES	1048576	1 MB body cap
AUTO_SEED	true	seed identity on empty DB
DEMO_SEED	false	true = boot the Acme \$902,900 sample

15. Deployment

- **Docker Compose** (docker-compose.yml): db + server + web. make up (live-only) / DEMO_SEED=1 make up (sample).
- **Render** (render.yaml) / **Vercel** (web/vercel.json) config present for hosted deploys.

- Health: `GET /health`, `GET /health/ready`.

16. Known constraints / deliberate simplifications

- Additive-only schema reconcile (no Alembic) — destructive changes need a DB recreate.
- In-memory rate-limit buckets + `_LAST_STATS` — per-process, reset on restart (fine for single-instance demo).
- URL analyzer offline (no provider keys) can over-flag some domains (structural signals only).
- `ThreatIntel` (Web3) model is a stub, not wired into v1 flows.